

PRODUCTION KNOWLEDGE. REAL ENGINEERING. BIGGER CAREERS.

AWS

POCKET GUIDE

FOR SENIOR ENGINEERS



ARCHITECTURE
AND DESIGN



SECURITY
AND COMPLIANCE



OPERATIONS
AND RELIABILITY



SCALING
AND PERFORMANCE



STORAGE
AND DATABASES



NETWORKING
AND CONNECTIVITY



COST OPTIMIZATION



REAL-WORLD
SCENARIOS



FROM DESIGN TO OPERATIONS.
REAL SYSTEMS. REAL SOLUTIONS.



EC2



S3



RDS



VPC



Lambda



EKS



ELB



CloudWatch

*Build
Operate
Scale
with Confidence*

ENGINEER A BETTER TOMORROW

SAME
CONCEPTS
REAL
PRODUCTION
VALUE

AWS Architecture Mental Model

Regions | Availability Zones | Edge Locations | Global vs Regional Services

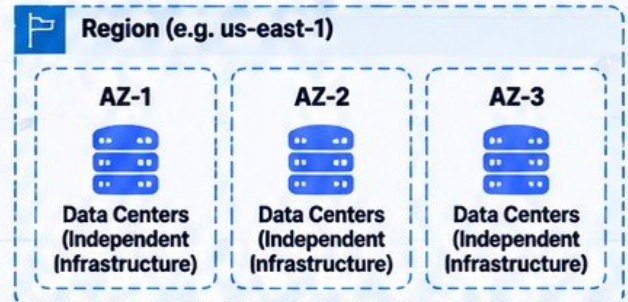
1. AWS Global Infrastructure



- Region
- Edge Location (CloudFront)
- Local Zone
- Wavelength Zone

- AWS operates a global infrastructure of Regions, Availability Zones, and edge locations.
- Choose the right location based on latency, compliance, data residency, and resilience requirements.

2. Region and Availability Zones



- A Region is a geographic area with multiple AZs.
- Each AZ has independent power, networking and cooling.
- AZs are connected with high-speed, low-latency links.
- Design for multi-AZ to achieve high availability.

3. Edge Locations



- Edge locations cache content close to users.
- Reduce latency and improve performance using Amazon CloudFront.
- Used for static content, dynamic acceleration and DDoS protection.
- Edge locations are not for running your applications.

4. Regional vs Global Services

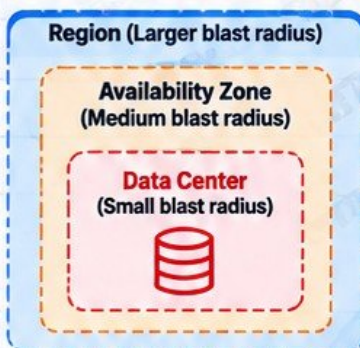
Regional Services (Deployed in a specific Region)

- EC2
- RDS
- VPC
- EKS
- S3 (Regional)

Global Services (Manage from anywhere)

- Route 53
- CloudFront
- IAM
- AWS Organizations
- AWS WAF

5. Failure Domains and Blast Radius



- Design to contain failures within the smallest possible blast radius.
- Use multi-AZ and multi-region for critical systems.
- Understand the impact of regional outages on your services and dependencies.

6. Key Takeaways

- ✓ Choose the right Region based on users, compliance and service availability.
- ✓ Use multiple AZs for high availability.
- ✓ Use edge locations (CloudFront) to reduce latency.
- ✓ Understand which services are regional or global.
- ✓ Design with failure domains and blast radius in mind.
- ✓ Map service dependencies across Regions and accounts.
- ✓ Always test your architecture assumptions.



Production Insight:

A well-designed architecture limits blast radius, survives failures, and keeps your users online.

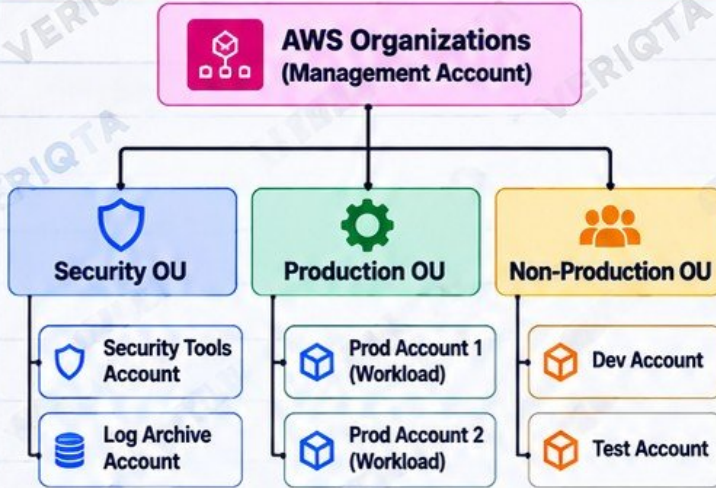


Remember: A single AWS service or Region failure can impact many dependent services.

AWS Account and Multi-Account Architecture

AWS Organizations | OUs | Account Boundaries | SCPs | Workload Isolation | Shared Services

1. AWS Organizations Structure



Organize accounts by purpose, apply guardrails, and isolate workloads.

2. Key Components

- AWS Organizations**
Centrally manage multiple AWS accounts.
- Organizational Units (OUs)**
Group accounts and apply policies.
- Service Control Policies (SCPs)**
Set guardrails (allow or deny actions).
- Account Boundaries**
Provide isolation and limit blast radius.
- Shared Services**
Centralize common services (logging, security).
- Workload Isolation**
Separate prod, non-prod, security and tooling.

3. SCP Example (Deny Risky Actions)

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Deny",
      "Action": ["ec2:TerminateInstances"],
      "Resource": "*"
    }
  ]
}
  
```

Note:
SCPs set the maximum permissions for accounts. They do not grant permissions. Use SCPs to prevent high-risk actions across OUs.

4. Production vs Non-Production

Production	Non-Production
✓ Strict access controls	✓ Separate accounts
✓ SCP guardrails	✓ More flexible access
✓ Dedicated accounts	✓ Lower cost structures
✓ Monitored and audited	✓ Used for dev, test, staging
✓ Change approval process	✓ Can use relaxed SCPs
✓ High availability design	✓ Should still have guardrails
✓ Lower blast radius	✓ Isolated from production
✓ Tagged and cost tracked	✓ Auto-expire unused resources

5. Shared Services Account (Example)



Best Practice:

Use a shared services account for centralized logging, security and networking services.

6. Key Takeaways

- ✓ Use AWS Organizations for multi-account environments.
- ✓ Design clear OU structure (prod, non-prod, security, shared).
- ✓ Apply SCPs to enforce guardrails and reduce risk.
- ✓ Isolate workloads to contain blast radius.
- ✓ Centralize common services in shared accounts.
- ✓ Use tagging, monitoring and cost controls.
- ✓ Regularly review account access and policies.
- ✓ Plan for failure scenarios at the account level.

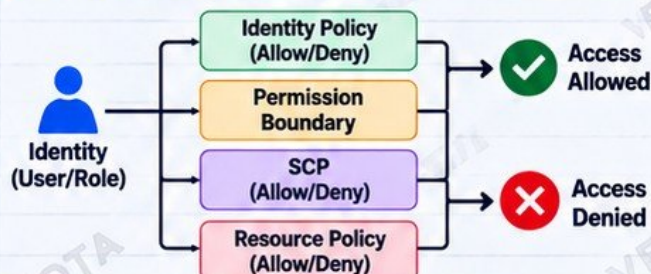
IAM and Authorization Deep Dive

Identity Policies | Resource Policies | Permission Boundaries | SCPs | Trust Policies | STS | Explicit Deny

1. IAM Policy Types and Scope

	Identity Policy	Attached to users, groups or roles. Defines what the principal can do.
	Resource Policy	Attached to resources (e.g. S3, SNS). Defines who can access the resource.
	Permission Boundary	Sets the maximum permissions an identity can have.
	SCP (Organizations)	Sets guardrails at the account/OU level. Denies or allows actions.
	Trust Policy	Defines who can assume a role (e.g. users, other accounts, AWS services).

2. How Authorization Works



Final decision = If any policy explicitly denies, the request is DENIED (Explicit Deny always wins).

3. STS and Temporary Credentials



AWS STS

- Provides temporary credentials.
- Used for role assumption.
- Improves security (short-lived access).
- Common in cross-account and federated access.

Example: Assume a role

```
aws sts assume-role \
--role-arn arn:aws:iam::123456789012:role/ProdRole \
--role-session-name veriqa-session
```

4. Trust Policy Example (Role Assumption)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "AWS": "123456789012"
      },
      "Action": "sts:AssumeRole"
    }
  ]
}
```

Use trust policies to:

- Allow specific AWS accounts, users, roles or services to assume a role.
- Enable cross-account access securely.
- Use conditions (e.g. MFA, external ID) for additional security.

5. Explicit Deny Example

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Deny",
      "Action": "s3:DeleteObject",
      "Resource": "arn:aws:s3:::my-bucket/*"
    }
  ]
}
```

Explicit Deny overrides any Allow from identity, resource, permission boundary or SCP.

6. Authorization Troubleshooting

- 1 Check the error message (AccessDenied).
- 2 Use IAM Policy Simulator to test permissions.
- 3 Verify all applicable policies (identity, resource, permission boundary, SCP).
- 4 Look for explicit deny statements.
- 5 Check if you are using the correct IAM principal.
- 6 Verify session context (STS, MFA, external ID).
- 7 Review CloudTrail for the denied request.

7. Common Authorization Scenarios

Scenario	Solution
User cannot access S3	Check identity policy, bucket policy and SCP.
Role assumption fails	Check trust policy and principal permissions.
Action denied unexpectedly	Look for explicit deny in SCP or permission boundary.
Cross-account access issues	Verify trust policy, external ID and resource policy.
Works for one user but not another	Compare their policies and group membership.

8. Key Takeaways

- ✓ Use least privilege.
- ✓ Understand all policy types and their evaluation order.
- ✓ Explicit deny always wins.
- ✓ Use STS and temporary credentials for better security.
- ✓ Apply permission boundaries and SCPs for guardrails.
- ✓ Use trust policies to control who can assume roles.
- ✓ Use CloudTrail and policy simulator for troubleshooting.



Production Insight:

Design your IAM strategy early. It prevents security incidents, reduces blast radius and simplifies operations at scale.

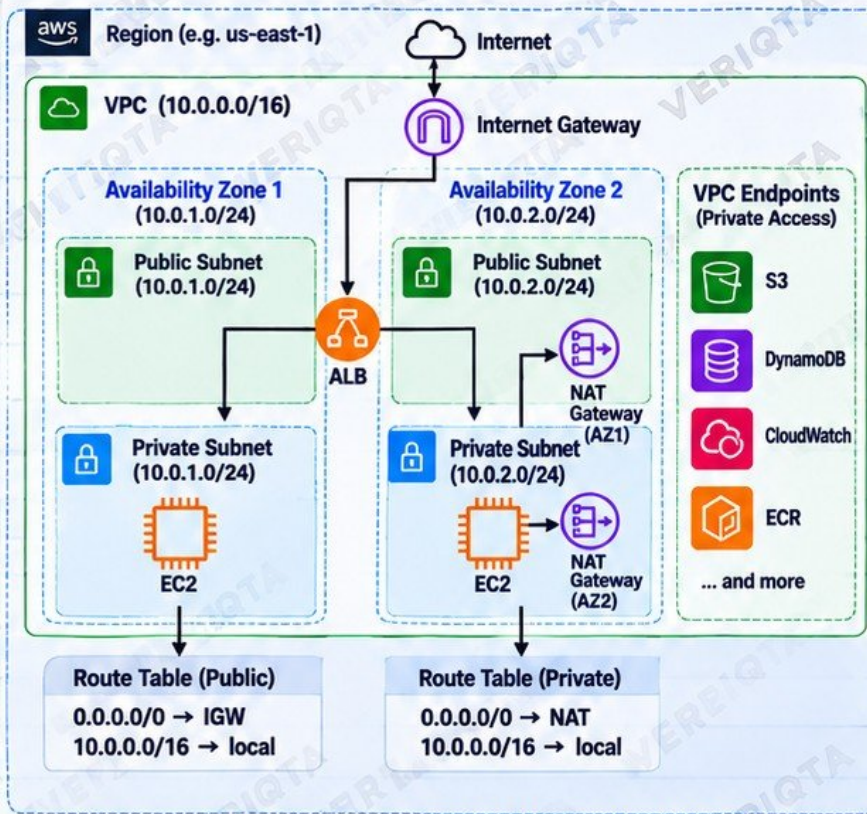


Avoid using * (wildcard) permissions in production environments.

Production VPC Architecture

CIDR Planning | Subnets | Route Tables | Internet Gateway | NAT Gateway | NACLs | Security Groups | VPC Endpoints | Multi-AZ

1. VPC Architecture (Multi-AZ)



2. Key Components

- VPC**
Isolated network in AWS.
- CIDR**
Plan IP address range (e.g. 10.0.0.0/16).
- Subnets**
Public (internet access) and Private (no direct internet access).
- Route Tables**
Control traffic flow between subnets.
- Internet Gateway (IGW)**
Provides internet access to public subnets.
- NAT Gateway**
Allows private subnets to access the internet (outbound only).
- Security Groups**
Stateful firewall at instance level (allow rules).
- Network ACLs (NACLs)**
Stateless firewall at subnet level (allow and deny rules).
- VPC Endpoints**
Private access to AWS services (no internet).
- Multi-AZ Design**
Deploy across multiple AZs for high availability.

3. CIDR Planning

- Choose a CIDR range that allows future growth.
- Use private IP ranges (e.g. 10.0.0.0/16, 172.16.0.0/16).
- Divide into subnets per AZ and purpose.
- Leave space for new subnets (e.g. add /20 or /24 blocks).

Example:

VPC: 10.0.0.0/16

Public Subnets: 10.0.1.0/24, 10.0.2.0/24

Private Subnets: 10.0.11.0/24, 10.0.12.0/24

4. Public vs Private Subnets

Public Subnet

- Has a route to IGW
- Used for ALB, bastion hosts
- Can have public IPs
- Holds internet-facing resources

Private Subnet

- No direct route to IGW
- Uses NAT for outbound access
- Holds application and database servers
- More secure (not publicly reachable)

5. Security Groups vs NACLs

Security Groups (Stateful)

- Instance level
- Allow rules only
- Stateful (return traffic automatically allowed)
- Easier to manage
- Use for application access control

Network ACLs (Stateless)

- Subnet level
- Allow and deny rules
- Stateless (must allow both directions)
- Used for an extra layer of defense
- Useful for blocking unwanted traffic

6. VPC Endpoints (Private Access)

- Access AWS services without internet.
- Types: Gateway Endpoints (S3, DynamoDB) and Interface Endpoints (most services).
- Improve security, reduce cost, and keep traffic private.

Example (S3 Gateway Endpoint Route):

Destination: pl-xxxxxxexxx (S3 prefix list)

Target: vpce-xxxxxxexxx

7. Multi-AZ Design Benefits

- Higher availability and fault tolerance
- Distributes resources across AZs
- Protects against AZ failure
- Use ALB, Auto Scaling, and Multi-AZ RDS
- Design for stateless applications
- Test failover and recovery regularly



Production Tip:
Design your VPC for scale, security and resilience from day one.
Changing IP ranges later is difficult.

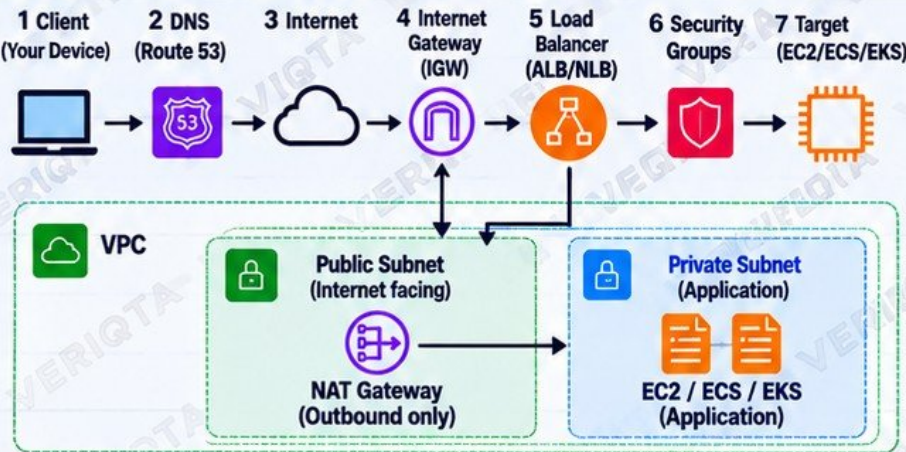
8. Common Mistakes

- Using overlapping CIDR ranges
- Placing databases in public subnets
- Missing NAT in private subnets
- Overly permissive security groups (0.0.0.0/0)
- Not using VPC endpoints
- Single-AZ deployment for critical workloads
- Ignoring NACLs
- Poor subnet planning (no room to grow)

AWS Network Traffic Troubleshooting

Follow a packet through DNS, routing, security groups, NACLs, NAT, load balancers, and application ports.
Diagnose timeout versus refusal versus DNS failure.

1. Packet Flow in a Typical AWS Architecture



2. Components to Check (In Order)

- 1 DNS Resolution (Route 53)
- 2 Internet Connectivity
- 3 Route Tables
- 4 Network ACLs (NACLs)
- 5 Security Groups
- 6 NAT Gateway (for outbound)
- 7 Load Balancers (ALB/NLB)
- 8 Target (EC2/ECS/EKS/other)
- 9 Application and Port
- 10 Return Traffic (stateful)

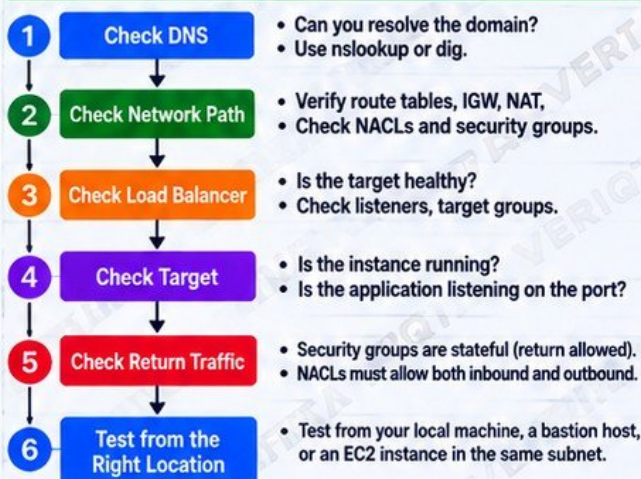
3. Common Network Issues and What They Mean

Issue / Symptom	Likely Cause	Where to Check
DNS failure ("could not resolve host")	DNS record missing or incorrect	Route 53, DNS settings, VPC DNS, resolver
Timeout (no response)	Traffic blocked (SG, NACL, route table, firewall)	Route tables, NACLs, SGs, NAT, IGW, firewall
Connection refused ("connection refused")	Service not listening on port or blocked by host firewall	Application, security group, instance, port configuration
HTTP 4xx/5xx	Application issue or misconfiguration	Load balancer, target health, application logs

4. Useful AWS CLI Commands

<code>nslookup example.com</code>	Test DNS resolution
<code>dig example.com</code>	Detailed DNS lookup
<code>aws ec2 describe-route-tables</code>	Check route tables
<code>aws ec2 describe-security-groups</code>	Check security groups
<code>aws ec2 describe-network-acls</code>	Check NACLs
<code>aws ec2 describe-nat-gateways</code>	Check NAT gateways
<code>aws elbv2 describe-load-balancers</code>	Check load balancers
<code>aws elbv2 describe-target-groups</code>	Check target health
<code>telnet <host> <port></code>	Test port connectivity
<code>curl -v http://<host></code>	Test application response

5. Troubleshooting Flow



Pro Tip:

Test from an EC2 instance in the same subnet for accurate results. It removes internet and external firewall variables.

6. Timeout vs Refusal vs DNS Failure

Type	What It Means	Common Causes	How to Fix
DNS Failure	Name cannot be resolved	Missing/incorrect DNS record, VPC DNS issues, resolver config	Fix Route 53 record, check VPC DNS settings
Timeout	No response received	Blocked by SG, NACL, route table, firewall, NAT issue, service down	Check network path, security groups, NACLs, routes, NAT, service health
Connection Refused	Host reachable but port closed	Service not running, wrong port, host firewall blocking	Start the service, use correct port, update SG or host firewall

7. Key Takeaways

- ✓ Always start with DNS and follow the packet step by step.
- ✓ Understand the difference between security groups (stateful) and NACLs (stateless).
- ✓ Use the right tools from the correct location.
- ✓ Check both inbound and outbound rules.
- ✓ Most timeouts are network issues, most refusals are application or port issues, and DNS failures are almost always name resolution problems.
- ✓ Document and fix the root cause, not just the symptom.

Route 53, DNS, and Traffic Management

Hosted zones | Records | Aliases | Health checks | Routing policies | TTL behavior | DNS caching | Failover | Production DNS incidents

1. What is Amazon Route 53?



- Amazon Route 53 is a highly available and scalable DNS and traffic management service.
- Translates domain names to IP addresses.
- Helps route traffic to healthy resources using different routing policies.
- Integrates with AWS and external resources.

2. Key Concepts



Domain
(Applies to your business)



Hosted Zone
(Contains DNS records)



Record
(Defines how a domain resolves)



Routing Policy
(Controls traffic flow)



Health Check
(Monitors resource health)

3. Hosted Zones

Public Hosted Zone



- Used for domains accessible on the internet.
- Example: veriqta.com
- Contains public DNS records (e.g. A, AAAA, CNAME, MX).

Private Hosted Zone



- Used for internal domains within a VPC.
- Not accessible from the internet.
- Example: internal.veriqta.local
- Used with Route 53 Resolver for internal DNS.

4. Common DNS Record Types

Type	Purpose	Example
A	Maps domain to IPv4 address	veriqta.com → 1.2.3.4
AAAA	Maps domain to IPv6 address	veriqta.com → 2600:1f18::1
CNAME	Alias to another domain	www → veriqta.com
Alias	AWS-specific alias (preferred)	www → ALB / CloudFront
MX	Mail server records	veriqta.com → mail server
TXT	Text records (verification, SPF)	veriqta.com → "v=spf1 ..."
NS	Name servers for the zone	veriqta.com → ns-xxxx
SOA	Start of authority (zone info)	Set automatically by AWS
SRV	Service records	_sip._tcp → server info

5. Alias Record vs CNAME

Alias Record (Recommended)

- ✓ AWS-specific (works with ELB, CloudFront, S3, etc.).
- ✓ No additional cost.
- ✓ Automatically updates to new IP addresses.
- ✓ Supports root domain (e.g. veriqta.com).

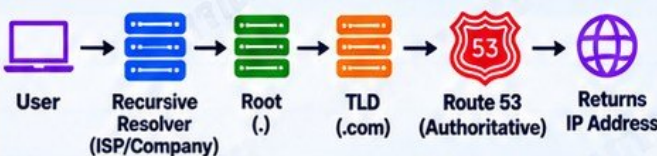
CNAME Record

- ✓ Points to another domain name.
- ✓ Cannot be used for root domain (e.g. veriqta.com).
- ✓ May not work with all AWS resources.
- ✓ Use when pointing to external domains.

6. Routing Policies

Policy	Use Case	Description
Simple	Single resource	Basic routing (default).
Weighted	Load distribution	Distribute traffic by weight.
Latency-based	Best performance	Route to the lowest latency Region.
Failover	High availability	Route to a healthy resource.
Geolocation	Country/region	Route based on user location.
Geoproximity	Proximity	Route based on geographic proximity (with bias).
Multivalued answer	Increase availability	Return multiple healthy IPs.

7. DNS Resolution Flow



8. Health Checks and Failover

Health Checks

- ✓ Monitor the health of endpoints (HTTP, HTTPS, TCP, etc.).
- ✓ Used with failover and multivalued routing.
- ✓ Can check external or AWS resources.
- ✓ Configurable failure threshold.

Failover Setup

- 1 Create a health check for primary.
- 2 Configure failover routing policy.
- 3 Set primary and secondary records.
- 4 Test by simulating failure.

9. TTL and DNS Caching

- TTL (Time To Live) controls how long a record is cached.
- Lower TTL (e.g. 60 seconds) = faster changes, higher query load.
- Higher TTL (e.g. 3600 seconds) = less query load, slower changes.
- DNS is cached by resolvers (ISP, local network, browser, OS).
- Changes may take time to propagate due to caching.

Common TTL Values

- 60 seconds (fast changes)
- 300 seconds (5 minutes)
- 3600 seconds (1 hour)
- 86400 seconds (1 day)

10. Common DNS Issues and How to Fix

Issue / Symptoms	Likely Cause	How to Fix
DNS not resolving	NXDOMAIN error	Check record exists, correct domain.
Wrong IP returned	Old/cached record	Wait for TTL or lower TTL.
Intermittent failures	Resolver caching	Flush local DNS cache.
Health check failing	Unhealthy endpoint	Check endpoint, security groups, routes.
Failover not working	Misconfigured records	Verify health check and routing policy.



Production Insight:

Use low TTL during migrations, then increase TTL after stability. Always test failover and monitor health check status.



Remember: DNS changes can take time to propagate due to caching. Always plan and test before production changes.

Elastic Load Balancing

ALB | NLB | Target Groups | Health Checks | TLS Termination | Connection Behavior | Cross-Zone | 4xx/5xx | Selection

1. What is Elastic Load Balancing?



- Distributes incoming traffic across multiple targets to improve availability, scalability and resilience.
- Automatically detects unhealthy targets and stops sending traffic to them.
- Supports different load balancer types for different use cases (ALB, NLB, GWLB).
- Integrates with EC2, ECS, EKS, Lambda and on-prem (via TGW).

2. Types of Load Balancers

Application Load Balancer (ALB)



- Layer 7 (HTTP/HTTPS)
- Path and host-based routing
- Supports WebSockets
- Best for web applications
- Integrates with WAF

Network Load Balancer (NLB)



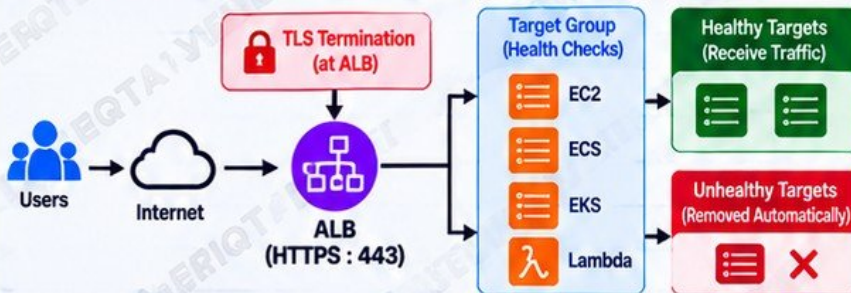
- Layer 4 (TCP/UDP/TLS)
- Ultra high performance
- Low latency
- Static IP addresses
- Best for non-HTTP traffic

Gateway Load Balancer (GWLB)



- Layer 3 (IP)
- Used for virtual appliances (firewalls, IDS/IPS)
- Transparent network traffic inspection

3. How a Load Balancer Works (ALB Example)



4. Target Groups



- Group of targets (EC2, ECS, EKS, Lambda, IP).
- Performs health checks on targets.
- Routes traffic only to healthy targets.
- Supports multiple target types and protocols.
- Configure health check path, port, interval, thresholds.
- Unhealthy targets are automatically removed.

5. Health Checks



- Determines if a target is healthy or unhealthy.
- Configurable path (e.g. /health), port, protocol, interval and timeout.
- Targets that fail the health check are marked unhealthy and stopped from receiving traffic.
- Use meaningful health check endpoints that reflect application health (not just /).

6. TLS Termination



- ALB and NLB can terminate TLS (HTTPS).
- Offloads SSL/TLS processing from targets.
- Use ACM (AWS Certificate Manager) for certificates.
- Supports SNI (multiple certificates).
- After termination, traffic to targets can be HTTP or HTTPS.

7. Connection Behavior



- ALB is connection-aware (Layer 7).
- NLB is connection-oriented (Layer 4).
- Supports HTTP keep-alive and WebSockets (ALB).
- NLB can handle millions of requests per second.
- Idle timeout and connection draining can be configured.

8. Cross-Zone Considerations



- By default, cross-zone load balancing is enabled across all AZs.
- Distributes traffic evenly across healthy targets in all enabled AZs.
- Disabling cross-zone can reduce data transfer costs but may lead to imbalance.
- Recommended: Keep it enabled for high availability.

9. 4xx and 5xx Diagnosis (ALB)



- 4xx errors: Client-side issues (bad request, unauthorized, not found).
- 5xx errors: Server-side issues (application errors, unhealthy targets).
- Check target health, application logs, and ALB access logs.
- Use CloudWatch metrics (HTTPCode_Target_4XX_Count, HTTPCode_Target_5XX_Count).
- Verify security groups, NACLs, and target configuration.

10. Load Balancer Selection Guide



Requirement	Best Choice
HTTP/HTTPS, path routing	ALB
Low latency, high throughput	NLB
Static IP addresses	NLB
Traffic inspection (firewalls)	GWLB
WebSocket support	ALB
Non-HTTP (TCP/UDP)	NLB
Layer 7 features (host/path)	ALB

11. Common Issues and Solutions

Issue / Symptom	Likely Cause	Solution
502 Bad Gateway	Unhealthy targets	Check target health, app logs
504 Gateway Timeout	Slow response from targets	Increase timeout, check app
503 Service Unavailable	No healthy targets	Verify health checks, scaling
Target not registered	Registration failure	Check TG configuration
SSL/TLS errors	Certificate or policy issue	Verify ACM cert and listener
High latency	Cross-region or app issue	Check target response time

12. Useful AWS CLI Commands

Command	Description
<code>aws elbv2 describe-load-balancers</code>	List load balancers
<code>aws elbv2 describe-target-groups</code>	List target groups
<code>aws elbv2 describe-target-health --target-group-arn <arn></code>	Check target health
<code>aws elbv2 describe-listeners</code>	List listeners
<code>aws elbv2 describe-rules</code>	View routing rules

13. Key Takeaways

- ✓ Choose the right load balancer for your use case.
- ✓ Use health checks and monitoring.
- ✓ Understand connection behavior and timeouts.
- ✓ Keep cross-zone enabled for resilience.
- ✓ Use TLS termination with ACM.
- ✓ Monitor 4xx/5xx and investigate early.
- ✓ Always test changes in non-production first.



Production Insight:

A load balancer is not just a traffic distributor, it is a critical control point for availability, security and performance. Monitor it like a core component.

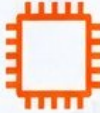


Remember: Always review ALB/NLB access logs, CloudWatch metrics and target health during incidents.

EC2 Production Engineering

Instance selection | AMIs | EBS | ENIs | User data | Metadata service | Placement | Auto Scaling | Spot | Instance failure | CPU and memory pressure | Operational debugging

1. What is Amazon EC2?

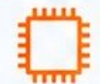


- Amazon EC2 provides resizable compute capacity in the cloud.
- You can choose the right instance type, storage, networking and scaling options for your workload.
- Used for web servers, application servers, batch processing, containers, databases and more.
- You are responsible for the OS, updates, monitoring and operations (thinking like a systems engineer).

2. Choose the Right Instance Type

Category	Use Case	Example
General Purpose	Balanced compute, memory, networking	t3, m7g
Compute Optimized	High CPU workloads	c7g, c7i
Memory Optimized	In-memory databases, caching	r7g, x2idn
Storage Optimized	High disk throughput	i4i, d3
Accelerated Computing	ML, GPU, high performance	p5, g5
HPC Optimized	High performance computing	hpc7g

3. EC2 Components



Instance
Virtual server in AWS



AMI
Pre-configured template



EBS
Block storage for the instance



ENI
Network interface



User Data
Runs scripts at launch



Metadata
Instance info (169.254.169.254)

4. Storage with EBS



- EBS volumes provide persistent block storage.
- Choose the right volume type: gp3 (general), io2 (high IOPS), st1/sc1 (throughput).
- You can attach, resize and take snapshots.
- Use multiple volumes for performance and separation of data.
- Enable encryption for sensitive data.

5. Networking with ENIs



- Each instance gets a primary ENI by default.
- You can attach additional ENIs.
- Each ENI can have multiple private IP addresses and security groups.
- Used for high availability, multi-homing and advanced architectures.

6. User Data (Example)

```
#!/bin/bash
yum update -y
yum install -y nginx
systemctl enable nginx
systemctl start nginx
echo "Hello from VERIQTA" > /usr/share/nginx/html/index.html
```

7. EC2 Metadata Service (IMDS)



- Provides instance metadata (e.g. instance ID, region).
- Use IMDSv2 for better security (token required).
- Access via 169.254.169.254.
- Used in automation, monitoring and configuration scripts.

8. Placement Options

Option	Description
Default	Instances placed by AWS (recommended).
Placement Group	Low latency or high throughput (same rack or same AZ).
Dedicated Host	Your instances run on a physical server (dedicated).
Dedicated Instance	Runs on hardware not shared with other customers.

9. Auto Scaling



- Automatically adjusts the number of instances.
- Responds to metrics (CPU, memory, custom metrics).
- Improves availability and handles traffic spikes.
- Works with Launch Templates and Load Balancers.

10. Spot Instances



- Use spare AWS capacity at lower cost.
- Ideal for fault-tolerant and flexible workloads.
- Instances can be interrupted (with short notice).
- Use Spot with Auto Scaling and fault-tolerant architectures.

11. Common Instance Failures



- Status check failures (system or instance).
- Out of memory (OOM).
- Disk full (EBS).
- Network connectivity issues.
- Kernel panic or OS crash.
- Spot instance interruption.
- Misconfiguration or failed user data.

12. CPU and Memory Pressure

Metric	What It Means	Action
High CPU	CPU > 80% consistently	Investigate process, scale or optimize.
High Memory	Memory > 80%	Check memory leaks, increase instance size.
Swap usage	Using swap space	Add memory or optimize application.
High load average	System overloaded	Check running processes, scale or fix application.

13. Operational Debugging (Linux)

Task	Command
Check CPU usage	top, htop, mpstat
Check memory usage	free -h, vmstat
Check disk usage	df -h, du -sh /var
Check processes	ps aux, top
Check network	ss -tulpn, netstat -tulpn
Check logs	journalctl -xe, tail -f /var/log/messages
Check instance metadata	curl http://169.254.169.254/latest/
Check instance status	aws ec2 describe-instance-status --instance-ids i-xxxxxx

14. Key Takeaways

- Choose the right instance type for your workload.
- Use AMIs and user data for consistent deployments.
- Monitor CPU, memory, disk and status checks.
- Use Auto Scaling and Spot for cost and resilience.
- Understand networking, storage and metadata.
- Be ready to troubleshoot and recover quickly.



Production Insight:

A well-engineered EC2 setup is not just about launching an instance, but about running, monitoring and operating it reliably at scale.



Remember:

Monitor, automate, and have a recovery plan. Instances will fail, be ready.

Auto Scaling and Elastic Capacity

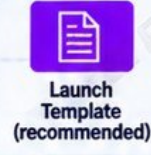
ASGs | Launch Templates | Target Tracking | Step Scaling | Lifecycle Hooks | Warm-up | Health Replacement | Scaling Lag | Capacity Shortages | Scaling Instability

1. What is Auto Scaling?



- Auto Scaling automatically adjusts the number of EC2 instances based on demand.
- Helps you maintain performance, availability, and cost efficiency.
- Uses Amazon EC2 Auto Scaling groups (ASGs).
- Works with metrics from CloudWatch, Application Load Balancers, and custom metrics.
- Part of building elastic and highly available systems.

2. Key Components



3. How Auto Scaling Works



4. Scaling Policies

Policy Type	Use Case	How It Works
Target Tracking	Keep metric at a target (e.g. CPU 50%)	Automatically adjusts capacity to maintain target.
Step Scaling	Scale based on metric thresholds	Adds/removes a fixed number or percentage.
Scheduled Scaling	Predictable traffic (e.g. business hours)	Scale at specific times (dates and times).

5. Launch Templates vs Launch Configurations

Launch Template (Recommended)

- ✓ More features and flexibility.
- ✓ Supports multiple versions.
- ✓ Use latest Amazon Linux, custom AMIs.
- ✓ Configure user data, instance type, security groups, IAM role, EBS, etc.
- ✓ Can be updated without replacing ASG.

Launch Configuration (Legacy)

- ✗ Older and limited features.
- ✗ No versioning.
- ✗ Still supported but not recommended for new workloads.
- ✗ Use launch templates instead.

6. Lifecycle Hooks



- Let you perform custom actions when instances launch or terminate.
- Common use cases:
 - Run scripts (e.g. register with Load Balancer)
 - Wait for application startup
 - Drain connections before termination
- Use a heartbeat timeout to control the waiting period.

7. Warm-up and Health Replacement

Warm-up Period

- Time for a new instance to be ready to handle traffic.
- Prevents premature scaling in or unhealthy status.
- Configure warm-up in scaling policies (especially with Target Tracking).

Health Checks and Replacement

- ASG monitors instance health via EC2 and/or ELB health checks.
- Unhealthy instances are automatically replaced.
- Helps maintain availability.
- Ensure your health check is properly configured.

8. Common Issues and Causes

Issue	Possible Cause	Solution
Scaling lag	Slow instance launch or long warm-up	Use correct instance type, optimize AMIs, adjust warm-up time.
Capacity shortage	Max capacity too low or Spot interruptions	Increase max capacity, use diversified instance types, consider On-Demand.
Scaling instability	Incorrect thresholds or conflicting policies	Review metrics, use Target Tracking, avoid multiple conflicting policies.
Instances not replaced	Health check misconfigured	Check ELB/EC2 health checks and instance status.

9. Example Scaling Policy (Target Tracking)

```
{
  "TargetValue": 50.0,
  "PredefinedMetricSpecification": {
    "PredefinedMetricType": "ASGAverageCPUUtilization"
  }
}
```

10. Best Practices

- ✓ Use launch templates.
- ✓ Choose the right min, desired and max capacity.
- ✓ Use Target Tracking for most workloads.
- ✓ Set appropriate warm-up periods.
- ✓ Monitor scaling activities and metrics.
- ✓ Test scaling regularly.
- ✓ Plan for Spot interruptions.
- ✓ Use lifecycle hooks for critical applications.



Key Takeaway

Auto Scaling gives you elastic capacity, but you must configure it correctly. Understand your workload, set sensible limits, monitor behavior, and test before you need it.

S3 Architecture and Data Protection

Object storage behavior | Consistency | Versioning | Lifecycle | Encryption | Access policies |
Block Public Access | Replication | Performance | Cost | Accidental deletion recovery

1. What is Amazon S3?



- Amazon S3 (Simple Storage Service) is highly durable, scalable object storage.
- Stores data as objects in buckets.
- Designed for 99.999999999% (11 9's) durability.
- Used for backups, data lakes, static websites, logs, media, ML/AI data and more.
- Pay only for what you use.

2. How S3 Stores Data (Object Model)



Example S3 URL
s3://veriqta-bucket/
images/logo.png

Objects can be
from 0 bytes to
5 TB in size.

3. Key S3 Features



Versioning
(Keep multiple versions)



Lifecycle
(Automate storage classes)



Encryption
(SSE-S3, SSE-KMS, Client-side)



Access Policies
(IAM, Bucket Policy, ACLs)



Block Public Access
(Prevent accidental exposure)



Replication
(Cross-Region/ Same-Region)



High Performance
(Designed for massive scale)



Cost Optimization
(Use lifecycle and right storage class)

4. Consistency Model



- S3 provides strong read-after-write consistency for all operations (including PUT, GET, DELETE, and LIST).
- Once you upload an object, you can immediately read it.
- Updates and deletes are also immediately consistent.
- No more eventual consistency for new objects (since Dec 2020).

5. S3 Storage Classes

Storage Class	Use Case	Cost	Retrieval Time
S3 Standard	Frequently accessed data	Higher	Milliseconds
S3 Intelligent-Tiering	Unknown or changing access	Optimized	Milliseconds
S3 Standard-IA	Infrequently accessed data	Lower	Milliseconds
S3 One Zone-IA	Non-critical, single AZ	Lower	Milliseconds
S3 Glacier Instant Retrieval	Long-term, rarely accessed	Low	Milliseconds
S3 Glacier Flexible Retrieval	Archive data	Very low	Minutes to hours
S3 Glacier Deep Archive	Long-term archive (compliance)	Lowest	Hours

6. Versioning and Deletion Recovery



- When versioning is enabled, every object version is saved.
- You can restore previous versions if objects are deleted or overwritten.
- Helps protect against accidental deletion and ransomware.
- You can also use MFA Delete for extra protection.
- Lifecycle rules can be used to expire old versions.

7. Encryption Options

Method	Description	Use Case
SSE-S3	AWS manages the keys (AES-256)	Default, easy to use
SSE-KMS	Uses AWS KMS for key management	More control, auditing
SSE-C	You manage the keys	Compliance requirements
Client-side	Encrypt before upload	Highest level of control

8. Access Control and Block Public Access



- Use IAM policies, bucket policies, or ACLs to control access.
- Enable Block Public Access (recommended) to prevent accidental public exposure.
- Follow the principle of least privilege.
- Use bucket policies with conditions (e.g. VPC, IP, AWS services).
- Regularly audit access with AWS CloudTrail and IAM Access Analyzer.

9. Replication



- Cross-Region Replication (CRR): replicate objects to another region (disaster recovery, compliance).
- Same-Region Replication (SRR): replicate within the same region (log aggregation, separation of concerns).
- Supports replica modification sync and replication of delete markers (with versioning).

10. Performance and Cost Tips



- S3 automatically scales to high request rates.
- Use multipart upload for large files.
- Use S3 Transfer Acceleration for global uploads.
- Choose the right storage class to reduce cost.
- Monitor usage with S3 Storage Lens.
- Use lifecycle policies to move or delete old data.

11. Common Issues and Solutions

Issue	Solution
Access Denied	Check IAM/bucket policy, Block Public Access
Object not found	Check key name and versioning
Slow uploads	Use multipart upload, check network
High cost	Use lifecycle, right storage class, review usage
Accidental deletion	Enable versioning, restore previous version
Public access risk	Enable Block Public Access, audit policies



Key Takeaway

S3 is simple but powerful. With the right configuration (versioning, encryption, access control, and lifecycle), you can protect your data, reduce cost, and recover from mistakes.



Remember: Enable versioning and Block Public Access on all new buckets unless you have a specific reason not to. Always test your recovery process.

EBS, EFS, and AWS Storage Decisions

Block versus shared file versus object storage | IOPS | Throughput | Latency | Snapshots | Multi-AZ implications | Attachment constraints | Storage bottleneck investigation

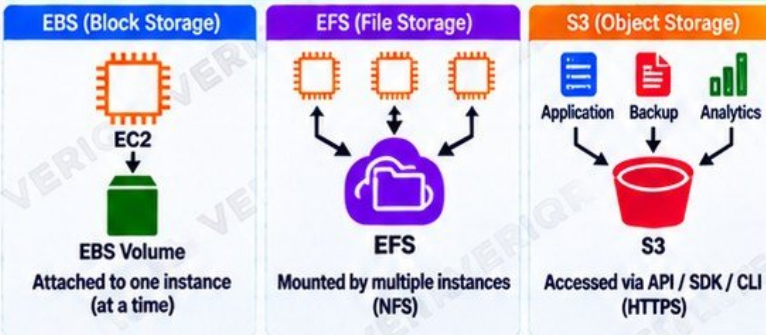
1. Storage Options in AWS

Amazon EBS (Block Storage)	Amazon EFS (File Storage)	Amazon S3 (Object Storage)
- Block storage for a single EC2 instance. - High performance and low latency. - Used for operating systems, databases and transactional workloads. - Not shared across multiple instances (single attach, Multi-Attach only for io2).	- Shared file storage (NFS). - Can be mounted by multiple EC2 instances at the same time. - Automatically scales. - Ideal for web servers, shared content, containers and big data. - Higher latency than EBS but easy to manage.	- Object storage for any amount of data. - Highly durable and scalable (11 9's). - Used for backups, media files, logs, data lakes and archival. - Accessed via API/SDK/CLI (not a file system). - Not mounted like a disk or file share.

2. Block vs File vs Object Storage

Feature	EBS (Block)	EFS (File)	S3 (Object)
Storage type	Block	File (NFS)	Object
Use case	EC2 OS, Databases	Shared files	Backups, logs, media, data lake
Access	Mounted to one EC2 instance (Multi-Attach for io2)	Mounted by multiple EC2 instances	Via API/CLI/SDK (HTTPS)
Latency	Low (milliseconds)	Higher (milliseconds)	Higher (10s of ms)
Performance	High IOPS/throughput	Scales automatically	High throughput for large objects
Scalability	Fixed per volume (can change)	Automatic	Virtually unlimited
Multi-AZ	Single AZ (replicated within AZ)	Multi-AZ by default	Multi-AZ by default
Durability	High (within AZ)	High (multi-AZ)	Very high (11 9's)
Cost	Higher	Moderate	Lowest (for large amounts)

3. Architecture Overview



4. IOPS, Throughput and Latency

Storage	Typical IOPS	Throughput	Latency
EBS gp3	3,000 (baseline)	125 MB/s (baseline)	Low (1-3 ms)
EBS io1/io2	Up to 256,000	Up to 4,000 MB/s	Very low (1 ms)
EFS (General)	Varies (scales)	Automatically scales	Low to moderate (5-10 ms)
S3 (Standard)	N/A (request based)	Very high	Higher (10s of ms)

5. Snapshots and Backup

EBS Snapshots	EFS Backup	S3 Versioning
- Point-in-time backup of EBS volumes. - Stored in S3 (incremental). - Can be copied across AZs and regions. - Used for disaster recovery and volume restore.	- Automatic backups with AWS Backup or EFS-to-EFS replication. - Supports point-in-time restore. - Can replicate across regions.	- Keep multiple versions of objects. - Protects against accidental deletion or overwrite. - Can enable lifecycle policies to move old versions to cheaper storage classes (e.g. S3 Glacier).

6. Attachment Constraints

- EBS:** Typically attached to one EC2 instance at a time (Multi-Attach supported for io2).
- EFS:** Can be mounted by many EC2 instances simultaneously.
- S3:** Not mounted. Access via API/CLI/SDK.

7. Multi-AZ Implications

Service	Multi-AZ Details
EBS	Automatically replicated within an AZ. Use EBS snapshots or Custom AMIs for cross-AZ/region DR.
EFS	Multi-AZ by default (stores data across multiple AZs).
S3	Multi-AZ by default (designed for high durability).

8. Investigating Storage Bottlenecks

- Check CloudWatch metrics (VolumeReadOps, VolumeWriteOps, BurstBalance, Throughput).
- Use iostat, df -h, and CloudWatch Agent on instances.
- Look for high queue length, high latency, or low throughput.
- Identify if the workload needs higher IOPS/throughput or a different storage type.
- For EFS, check burst credits and access patterns.
- For S3, check request rates, object size and network latency.

9. Storage Decision Guide

Use Case	Best Option
EC2 OS / Boot volume	EBS (gp3)
High performance DB	EBS (io1/io2)
Shared application files	EFS
Static files, backups, logs	S3
Large data archive	S3 (Glacier)
Low latency, high IOPS	EBS (io2)
Multi-instance file access	EFS
Global scalable storage	S3



Key Takeaway

Choose the right storage for the right workload. Understand performance, durability, cost and access requirements, and always plan for backup and recovery.



Remember: Monitor storage metrics, set up backups, and test your recovery process. Storage issues can impact application performance and availability.

RDS and Aurora in Production

Multi-AZ | Replicas | Endpoints | Connection Pools | Failover | Backups | Performance Insights | CPU Saturation | Locks | Replication Lag | Storage Pressure | Database Recovery

1. What is Amazon RDS and Aurora?

Amazon RDS



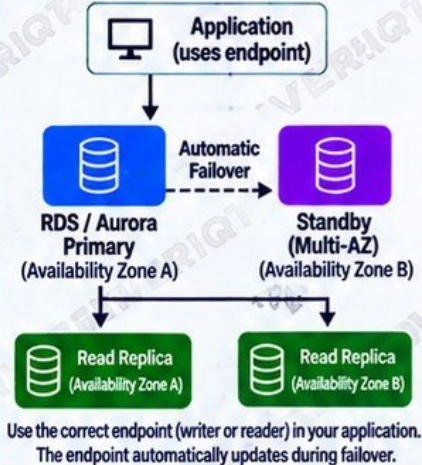
- Managed relational database service for MySQL, PostgreSQL, MariaDB, Oracle and SQL Server.
- Automates provisioning, patching, backups and monitoring.
- Supports Multi-AZ, Read Replicas and automated failover.
- Ideal for traditional relational databases.

Amazon Aurora



- AWS-designed, high-performance database (MySQL and PostgreSQL compatible).
- Higher performance and availability than standard RDS.
- Storage automatically scales (up to 128 TB).
- Faster failover (typically < 30 seconds).
- Designed for cloud-native applications.

2. High Availability Architecture



3. Endpoints

Endpoint Type	Use Case
Primary / Writer Endpoint	Used for read and write operations.
Reader Endpoint	Used for read-only operations (load balanced across replicas).
Cluster Endpoint (Aurora)	Used for read and write (automatically points to the primary).
Reader Endpoint (Aurora)	Used for read-only (load balanced across replicas).
Custom Endpoint	Route traffic to a specific subset of instances.

4. Connection Pools



- Use a connection pool to reuse database connections and reduce overhead.
- Recommended tools: RDS Proxy, PgBouncer, HikariCP, or application-level pooling.
- Helps prevent connection exhaustion during traffic spikes.
- Set appropriate max connections and timeouts.

5. Backups and Recovery



- Automatic backups (daily, 7-35 days).
- Enable point-in-time recovery (PITR).
- Take manual snapshots before major changes.
- Store snapshots in another region for disaster recovery.
- For Aurora, backups are continuous and incremental.
- Test your restore process regularly.

6. Performance Insights



- Visualize database load, wait events and SQL queries.
- Helps identify CPU, I/O, locks and slow queries.
- Available for RDS and Aurora.
- Use it to find bottlenecks and optimize performance.
- Enable enhanced monitoring for deeper visibility.

7. Common Production Issues

Issue	Possible Cause
CPU saturation	High queries, inefficient queries, missing indexes
High memory usage	Large queries, insufficient instance size
Locks	Long-running transactions, missing indexes
Replication lag	Heavy write load, large transactions
Storage pressure	Data growth, logs, snapshots, insufficient storage
High connection count	No connection pooling, application issue
Slow queries	Missing indexes, bad query design
Failover issues	Application not using the correct endpoint

8. Monitoring Key Metrics



Metric	What it Tells You
CPUUtilization	High CPU usage
FreeableMemory	Available memory
DatabaseConnections	Number of active connections
ReadLatency / WriteLatency	I/O performance
ReadIOPS / WriteIOPS	Disk I/O load
ReplicaLag	Replication delay
FreeStorageSpace	Remaining storage
SwapUsage	Memory pressure
Deadlocks	Lock contention
VolumeReadOps / VolumeWriteOps	Storage activity

9. Failover Process



1. Primary instance becomes unavailable (e.g. hardware failure).
2. Amazon automatically promotes the standby.
3. Endpoint is updated to point to the new primary.
4. DNS TTL is low (typically 5 seconds).
5. Applications using the endpoint reconnect automatically.
6. For Aurora, failover is usually faster (< 30 seconds).

10. Best Practices

- ✓ Use Multi-AZ for high availability.
- ✓ Use read replicas for read-heavy workloads.
- ✓ Use connection pooling (RDS Proxy or application-level).
- ✓ Monitor with Performance Insights and CloudWatch.
- ✓ Set up automated backups and test restores.
- ✓ Use encryption at rest and in transit.
- ✓ Plan for storage growth and set up alerts.
- ✓ Test failover and recovery procedures.

11. Database Recovery Scenarios



- Accidental deletion → Restore from snapshot or point-in-time recovery.
- Data corruption → Restore to a previous point in time.
- Human error → Use automated backups.
- Regional disaster → Restore from cross-region snapshot.
- Test recovery regularly to ensure it works.

12. Key Takeaways



- Use the right architecture (Multi-AZ, replicas).
- Monitor key metrics and set alarms.
- Use connection pools to avoid exhaustion.
- Regularly review performance and optimize.
- Have a backup and recovery plan.
- Understand failover behavior and endpoints.
- Investigate and resolve bottlenecks early.
- Test everything in a non-production environment.

Containers on AWS: ECS vs EKS vs Fargate

ECS vs EKS vs Fargate | ECR | Task and Pod Identity | Networking | Load Balancing | Image Security | Scaling | Deployment Behavior | Operational Ownership Trade-offs

1. What are Containers on AWS?



- Containers package your application, its dependencies and runtime into a single unit.
- AWS provides multiple ways to run containers: ECS, EKS and Fargate.
- You store container images in Amazon ECR (Elastic Container Registry).
- Containers help you build portable, scalable and consistent applications.

2. AWS Container Services

Amazon ECS



- Managed container service.
- Uses AWS native constructs.
- Simpler to operate.
- Works with EC2 or Fargate.
- Good for most workloads.

Amazon EKS



- Managed Kubernetes service.
- Industry standard (Kubernetes).
- More flexibility and larger ecosystem.
- Supports EC2 or Fargate.
- Ideal for complex and large-scale workloads.

AWS Fargate



- Serverless compute for containers.
- No servers to manage.
- Works with ECS or EKS.
- Pay only for what you use.
- Ideal for teams that want to run containers without managing EC2 instances.

3. ECR (Elastic Container Registry)



- Stores and manages your container images.
- Integrated with IAM for secure access.
- Supports image scanning for vulnerabilities.
- Use lifecycle policies to manage storage costs.
- Works with ECS, EKS and Fargate.

Example: Push an image to ECR

```
aws ecr get-login-password --region us-east-1 | \
docker login --username AWS --password-stdin 123456789012.dkr.ecr.us-east-1.amazonaws.com
docker build -t myapp .
docker tag myapp:latest 123456789012.dkr.ecr.us-east-1.amazonaws.com/myapp:latest
docker push 123456789012.dkr.ecr.us-east-1.amazonaws.com/myapp:latest
```

4. Task and Pod Identity



- ECS:** Use IAM roles for tasks (task roles and execution roles).
- EKS:** Use IAM roles for service accounts (IRSA).
- Fargate:** Use IAM roles (no instance profile needed).
- Gives containers secure access to AWS services without hardcoding credentials.

5. Networking and Load Balancing



- ECS:** Uses AWS VPC, ENIs, security groups and ALB/NLB.
- EKS:** Uses VPC, CNI plugins, Kubernetes services, ALB/NLB (via AWS Load Balancer Controller).
- Fargate:** Integrates with VPC (awsvpc mode) and works with ALB/NLB.
- Use security groups and network policies to control traffic.

6. Comparison: ECS vs EKS vs Fargate

Feature	ECS	EKS	Fargate
Orchestration	AWS native	Kubernetes	AWS managed (serverless)
Control plane	Managed by AWS	Managed by AWS	Managed by AWS
Compute options	EC2 or Fargate	EC2 or Fargate	Serverless only
Complexity	Simple	More complex	Very simple
Best for	General workloads	Complex, large-scale	Teams that don't want to manage servers
Cost	Lower (more control)	Higher (more features)	Pay per vCPU and memory
Ecosystem	AWS focused	Large ecosystem	AWS native
Learning curve	Easier	Steeper	Easiest

7. Scaling and Deployment Behavior



- ECS:** Service Auto Scaling (based on CPU, memory, custom metrics).
- EKS:** Horizontal Pod Autoscaler (HPA) and Cluster Autoscaler.
- Fargate:** Scales tasks/pods automatically (no EC2 to manage).
- Deployments:** rolling updates, blue/green, canary (using CodeDeploy, ArgoCD, or native tools).
- Use health checks and readiness probes to avoid downtime.

8. Image Security



- Use ECR image scanning to detect vulnerabilities.
- Keep base images updated (use minimal images).
- Use signed images (e.g. AWS Signer, cosign).
- Apply least privilege with IAM.
- Scan images in CI/CD pipeline before deployment.

9. Operational Ownership Trade-offs

Aspect	ECS	EKS	Fargate
Infrastructure mgmt	Low	Medium	None
Flexibility	Medium	High	Medium
Operational effort	Low	High	Very low
Customization	Medium	High	Limited
Best fit	Simpler workloads	Complex workloads	Serverless containers

10. Common Use Cases

- ECS:** Web applications, microservices, batch jobs.
- EKS:** Large microservices, multi-cloud, advanced Kubernetes features.
- Fargate:** Event-driven apps, variable workloads, no server management.
- ECR:** Store images for all container workloads.

11. Key Takeaways



- Choose the right service for your needs.
- Use ECR and scan your images.
- Implement secure identity (task roles or IRSA).
- Design proper networking and load balancing.
- Plan for scaling and zero-downtime deployments.
- Understand operational trade-offs.
- Keep containers secure, updated and monitored.



Remember: There is no one-size-fits-all. ECS, EKS and Fargate are all powerful. Choose based on your workload, team skills and operational requirements.



Production Tip: Start simple, automate everything, and always monitor your containers.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

ENGINEER A BETTER TOMORROW



Serverless and Event-Driven

AWS Lambda | API Gateway | SQS | SNS | EventBridge | Concurrency | Retries | DLQs | Idempotency | Throttling | Event Loss | Duplicate Processing | Asynchronous Failure Handling

1. What is Serverless and Event-Driven?



- Serverless lets you run code without managing servers (AWS Lambda).
- Event-driven systems react to events (requests, messages, state changes).
- You build scalable, cost-efficient and highly available applications by connecting AWS services.
- You only pay for what you use.
- AWS automatically handles infrastructure, scaling and high availability.

2. Key AWS Services



AWS Lambda
Run code without servers



API Gateway
Expose APIs (HTTP/REST)



SQS
Reliable message queue



SNS
Pub/Sub messaging



EventBridge
Event routing and integration

3. Typical Serverless Architecture



4. Lambda Concurrency and Scaling



- Lambda scales automatically based on demand.
- Each function has a concurrency limit (default 1,000).
- Reserve concurrency to prevent throttling and protect critical functions.
- Monitor with CloudWatch (Invocations, Errors, Throttles, Duration).
- Use provisioned concurrency for consistent low latency.

5. Retries and Dead Letter Queues (DLQs)



- AWS automatically retries failed Lambda executions (for asynchronous events).
- Use DLQs (SQS or SNS) to capture events that still fail after all retries.
- Configure maximum retry attempts and event age (e.g. 3 retries, 6 hours).
- Always monitor DLQs and set up alerts.

6. Idempotency



- Design your functions to be idempotent (safe to run multiple times).
- Use unique IDs (e.g. orderId, eventId) to track processed events.
- Store state in DynamoDB or a database to prevent duplicate processing.
- Idempotency is critical because events can be delivered more than once (at-least-once delivery).

7. Throttling and Limits



- Throttling occurs when request rate exceeds service limits.
- You may see 429 Too Many Requests or ThrottlingException.
- Use exponential backoff and retries.
- Request service limit increases if needed.
- Use SQS to buffer traffic and smooth spikes.

8. Common Issues and Troubleshooting

Issue	Possible Cause	Solution
Function throttled	High traffic, concurrency limit	Increase limits, use SQS, enable reserved concurrency
Event lost	Misconfigured permissions or event source	Check IAM roles, event source settings, CloudWatch logs
Duplicate processing	At-least-once delivery	Implement idempotency, use unique IDs
Messages in DLQ	Function errors, invalid data	Check logs, fix code, reprocess messages
High latency	Cold starts, external dependencies	Use provisioned concurrency, optimize code
API Gateway errors	Invalid request, Lambda error	Check CloudWatch logs and API Gateway logs

9. Asynchronous Failure Handling



- Use DLQs for failed events.
- Set up CloudWatch alarms for errors, throttles and DLQ messages.
- Implement monitoring and alerting.
- Build replay mechanisms to reprocess failed events from DLQ.
- Use EventBridge for event retries and failure handling.

10. Best Practices

- ✓ Use least privilege IAM roles.
- ✓ Implement idempotency.
- ✓ Use DLQs and monitor them.
- ✓ Set appropriate retry and timeout values.
- ✓ Use SQS to handle spikes and decouple services.
- ✓ Monitor with CloudWatch and set alarms.
- ✓ Test failure scenarios (throttling, retries, DLQ).
- ✓ Keep functions small and focused.

11. Use Cases



- Web and mobile backends (API Gateway + Lambda).
- Background processing (SQS + Lambda).
- Event-driven microservices (EventBridge).
- Real-time notifications (SNS).
- File processing (S3 event → Lambda).
- Data pipeline orchestration.
- Scheduled tasks (EventBridge).

12. Key Takeaway



Serverless and event-driven architectures are powerful, scalable and cost efficient, but you must design for failures, duplicates and throttling. Always monitor, use DLQs and implement idempotency.

AWS Observability and Operational Evidence

CloudWatch metrics | logs | alarms | dashboards | CloudTrail | tracing | VPC Flow Logs | application telemetry | cardinality | retention | evidence-driven incident investigation

1. What is Observability on AWS?



- Observability is the ability to understand the health, performance and behavior of your systems using metrics, logs and traces.
- Helps you detect, troubleshoot and resolve issues faster.
- Provides operational evidence during incidents, audits and post-incident reviews.
- **Key services:** CloudWatch, CloudTrail, X-Ray, VPC Flow Logs and application telemetry.

2. Key AWS Observability Services



CloudWatch
Metrics, logs, alarms, dashboards



CloudTrail
API activity and audit logs



X-Ray
Distributed tracing



VPC Flow Logs
Network traffic in and out of resources



Application Telemetry
Logs, metrics and traces from your applications

3. CloudWatch Metrics, Logs, Alarms and Dashboards



Metrics

- Predefined and custom metrics
- Monitor CPU, memory, errors, latency, etc.
- Use namespaces (AWS and custom)



Logs

- Collect logs from AWS services, EC2, containers, applications
- Use CloudWatch Logs agent or AWS integrations
- Set retention period



Alarms

- Trigger alerts based on metrics
- Send notifications via SNS
- Use anomaly detection and composite alarms



Dashboards

- Visualize metrics, logs and alarms
- Create custom dashboards for operations
- Use for real-time monitoring and incident response

4. CloudTrail and VPC Flow Logs



CloudTrail

- Records AWS API activity (who did what, when, from where)
- Stored in S3 (can also send to CloudWatch Logs)
- Critical for security, compliance and incident investigation
- Monitor for unauthorized or unexpected changes



VPC Flow Logs

- Captures IP traffic to and from your VPC (ENIs, subnets, etc.)
- Helps with network troubleshooting and security analysis
- Send to CloudWatch Logs or S3
- Use with tools like Athena to analyze traffic

5. Tracing with AWS X-Ray



- Provides end-to-end tracing of requests across microservices.
- Helps identify latency, errors and bottlenecks.
- Integrates with Lambda, ECS, EKS and API Gateway.
- Visualize service map and trace details.

6. Application Telemetry



- Collect logs, metrics and traces from your applications.
- Use OpenTelemetry, AWS Distro for OpenTelemetry (ADOT) or language-specific SDKs.
- Instrument key business transactions.
- Monitor errors, latency and user behavior.

7. Cardinality and Retention



- **Cardinality:** The number of unique values in your metrics (e.g. user IDs, IP addresses).
- High cardinality can increase cost and reduce performance.
- Use meaningful, low-cardinality dimensions.
- Set appropriate log and metric retention to balance cost and compliance (e.g. 7, 30, 90, 365 days).

8. Evidence-Driven Incident Investigation



- 1 Identify the impact (what is affected).
- 2 Check CloudWatch dashboards and alarms.
- 3 Analyze logs (CloudWatch Logs, application logs).
- 4 Review traces (X-Ray) for latency and errors.
- 5 Check CloudTrail for recent changes.
- 6 Use VPC Flow Logs to investigate network issues.
- 7 Correlate evidence across all sources and build a timeline.

9. Common Issues and Solutions

Issue	Solution
Missing logs	Check log agent, IAM permissions, log group and retention
High metric cardinality	Use fewer dimensions, avoid unique IDs
No alarms triggering	Check metric name, threshold and evaluation period
Incomplete traces	Verify instrumentation and sampling settings
High observability cost	Set proper retention, filter logs, use metric filters
Insufficient evidence during incident	Enable CloudTrail, VPC Flow Logs and application logging early

10. Best Practices

- ✓ Monitor what matters (golden signals).
- ✓ Set meaningful alarms and alerting.
- ✓ Centralize logs and use structured logging.
- ✓ Enable CloudTrail in all accounts.
- ✓ Use X-Ray or OpenTelemetry for tracing.
- ✓ Control cardinality and manage retention.
- ✓ Automate evidence collection for incidents.
- ✓ Regularly review dashboards and costs.
- ✓ Test your monitoring and alerting.



Pro Tip: Good observability turns a 5-hour incident into a 5-minute investigation.



Remember: You cannot investigate what you do not log. Enable logging and monitoring before you need it.

AWS Security Architecture

KMS | Secrets Manager | IAM Roles | Workload Identity | GuardDuty | Security Hub | WAF | Encryption | Credential Rotation | Audit Trails | Network Segmentation | Least Privilege

1. Security Architecture Overview



- Protect data, workloads and users across your AWS environment.
- Use IAM, encryption, network controls and monitoring together.
- Follow the principle of least privilege.
- Detect and respond to threats quickly.
- Maintain audit trails and compliance.

Security is a shared responsibility between AWS and you.

2. Key AWS Security Services



AWS KMS
Key management and encryption



AWS Secrets Manager
Store and rotate secrets



AWS IAM
Users, groups, roles and policies



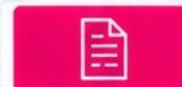
Amazon GuardDuty
Threat detection



AWS Security Hub
Central security findings



AWS WAF
Web application firewall



AWS CloudTrail
Audit logs and API activity



VPC
Network segmentation

3. AWS KMS (Key Management Service)



- Create and manage encryption keys (CMKs).
- Encrypt data at rest and in transit.
- Used by services like S3, EBS, RDS, Secrets Manager.
- Control access with IAM policies and key policies.
- Enable key rotation (automatic or manual).
- Monitor usage through CloudTrail.

4. AWS Secrets Manager



- Store sensitive information (database credentials, API keys, etc.).
- Automatic credential rotation.
- Fine-grained access control with IAM.
- Integrates with RDS, Lambda, ECS, EKS.
- Monitor access through CloudTrail.
- Use instead of hardcoding secrets in your code.

5. IAM Roles and Workload Identity



- Use IAM roles instead of long-term access keys.
- Assign least privilege permissions.
- Use service-linked roles for AWS services.
- Implement workload identity (ECS task roles, EKS IRSA).
- Use conditions in IAM policies (e.g. source IP, VPC, tags).
- Regularly review and remove unused permissions.

6. Amazon GuardDuty



- Threat detection for your AWS environment.
- Identifies unusual behavior, malware, unauthorized access and more.
- Uses machine learning and threat intelligence.
- No infrastructure to manage (fully managed).
- Sends findings to Security Hub and EventBridge.
- Investigate findings and take action quickly.

7. AWS Security Hub



- Centralizes security findings from multiple AWS services (GuardDuty, Inspector, Macie, etc.).
- Provides a single dashboard for security posture.
- Helps with compliance (CIS, PCI, SOC, etc.).
- Prioritize and track remediation of security issues.

8. AWS WAF (Web Application Firewall)



- Protects web applications from common attacks (SQL injection, XSS, etc.).
- Works with CloudFront, ALB and API Gateway.
- Use managed rule groups (AWS and third-party).
- Create custom rules for your needs.
- Monitor and tune rules to reduce false positives.

9. Encryption in AWS



- Encrypt data at rest (S3, EBS, RDS, EFS, etc.).
- Encrypt data in transit (TLS/HTTPS).
- Use AWS KMS for key management.
- Enforce encryption using IAM policies and bucket policies.
- Regularly review encryption settings.
- Ensure logs and backups are also encrypted.

10. Network Segmentation



- Use VPCs, subnets and route tables to isolate resources.
- Separate public and private subnets.
- Use security groups (instance-level firewall).
- Use network ACLs (subnet-level firewall).
- Implement VPC endpoints to access AWS services privately.
- Follow the principle of least exposure.

11. Audit Trails and Monitoring



- Use AWS CloudTrail to log all API activity.
- Store logs in a centralized S3 bucket (with encryption).
- Enable log file integrity validation.
- Monitor logs with CloudWatch and Security Hub.
- Keep logs for compliance and forensic investigations.
- Set up alerts for suspicious activity.

12. Credential Rotation and Best Practices



- Enable automatic rotation for database credentials.
- Rotate IAM access keys regularly.
- Use Secrets Manager for application secrets.
- Remove unused users, keys and roles.
- Enforce MFA for human users.
- Follow the principle of least privilege everywhere.

13. Common Security Issues



- Overly permissive IAM policies.
- Unencrypted data.
- Exposed credentials in code.
- Publicly accessible resources.
- Missing logging and monitoring.
- Unsegmented networks.
- No rotation for long-lived credentials.
- Unmonitored security findings.

14. Key Takeaways



- Use least privilege access.
- Encrypt everything.
- Use managed security services.
- Segment your network.
- Rotate credentials regularly.
- Monitor, log and review.
- Be proactive, not reactive.
- Security is an ongoing process.

Infrastructure as Code and Change Safety

Terraform and CloudFormation Concepts | State | Drift | Plans | Replacement Risk | Immutable Infrastructure | CI/CD Controls | Approvals | Rollback | Dangerous Infrastructure Changes

1. What is Infrastructure as Code (IaC)?



- IaC lets you define and manage infrastructure using code instead of manual clicks.
- Improves consistency, repeatability and version control.
- Reduces human error and enables automated deployments.
- Two popular tools on AWS: Terraform and AWS CloudFormation.
- Essential for scalable, secure and reliable infrastructure.

Infrastructure as Code = versioned, repeatable, reviewable and predictable infrastructure.

2. Terraform vs CloudFormation



- Multi-cloud (not just AWS).
- Uses HashiCorp Configuration Language (HCL).
- Stores state in a backend (e.g. S3 + DynamoDB).
- Uses terraform plan and terraform apply.
- Large community and many providers.



- AWS native service.
- Uses JSON or YAML.
- No separate state file (managed by AWS).
- Uses change sets for preview.
- Deep integration with AWS services.
- Ideal if you are AWS-only.

3. Key Concepts

Configuration	Code that defines resources (e.g. VPC, EC2, RDS).
State	Tracks what is deployed (Terraform) or managed (CloudFormation).
Drift	Difference between actual infrastructure and code.
Plan / Change Set	Preview of changes before applying.
Apply / Deploy	Creates, updates or deletes resources.
Replacement	Some changes require resource recreation (e.g. changing instance type).
Immutable Infra	Replace instead of modify for safer, predictable deployments.

4. Terraform State Management



- State file tracks the real infrastructure.
- Store state remotely (e.g. S3) and use DynamoDB for state locking.
- Never store state files locally in production.
- Protect state with encryption and strict IAM permissions.
- Use workspaces for multiple environments.
- If state is lost or corrupted, you can accidentally destroy resources.

5. Detecting and Handling Drift



- Drift happens when infrastructure is changed outside of IaC (e.g. manual changes in AWS console).
- Use terraform plan or CloudFormation drift detection.
- Regularly check for drift in all environments.
- Investigate and reconcile unexpected changes.
- Use strict IAM policies to prevent manual changes.

6. Plans, Change Sets and Approvals



- Always review terraform plan or CloudFormation change sets.
- Understand what will be created, changed or destroyed.
- Use pull requests and code reviews (e.g. GitHub) for all changes.
- Implement approval workflows (e.g. in CI/CD pipelines).
- Never apply changes directly in production without review.

7. Replacement Risk



- Some changes force resource replacement (e.g. changing subnet, instance type, or encryption settings).
- Replacement can cause downtime if not planned.
- Use lifecycle rules in Terraform to prevent unwanted replacements.
- Understand dependencies before making changes.
- Test changes in a non-production environment first.

8. Immutable Infrastructure



- Instead of modifying existing resources, create new ones.
- Reduces configuration drift and manual fixes.
- Works well with auto scaling and blue/green deployments.
- Easier rollbacks and more stable environments.
- Use IaC + CI/CD to automate immutable deployments.

9. CI/CD Controls for IaC



- Store IaC code in Git (e.g. GitHub).
- Use automated checks (linting, security scanning, policy checks).
- Run terraform plan or change set in CI.
- Require manual approval for production environments.
- Use separate pipelines for dev, staging and production.
- Keep audit logs of all changes.

10. Rollback Strategies



- Keep previous versions of your code.
- Use version control (Git) to revert changes.
- For Terraform, use terraform apply with a previous commit.
- For CloudFormation, use previous template or stack rollback.
- Test rollback procedures regularly.
- Have a documented disaster recovery plan.

11. Dangerous Infrastructure Changes



- Deleting critical resources (e.g. VPC, subnets, RDS, S3 buckets).
- Changing security groups or NACLs (this can lock you out).
- Modifying IAM roles and policies.
- Replacing databases or load balancers.
- Disabling encryption or logging.
- Always double-check plans, use approvals and test in non-production first.

12. Best Practices



- Use version control for all IaC code.
- Store Terraform state in S3 with DynamoDB locking.
- Enable drift detection regularly.
- Use pull requests and mandatory reviews.
- Implement automated testing and security checks.
- Follow least privilege IAM permissions.
- Prefer immutable infrastructure.
- Document all changes and maintain audit trails.
- Have a clear rollback plan.
- Never make direct manual changes in production.



Remember:
Infrastructure changes can impact availability, security and cost. Validate, review and test before you deploy.



Pro Tip:
Automate as much as possible, but always keep humans in the loop for production changes.

High Availability and Disaster Recovery

Multi-AZ vs Multi-Region | Backup and Restore | Pilot Light | Warm Standby | Active-Passive | Active-Active | RTO | RPO | Dependency Mapping | Failover | Recovery Testing

1. What is High Availability (HA) and Disaster Recovery (DR)?



- **High Availability (HA):** Keeps your application running during failures (minimal downtime).
- **Disaster Recovery (DR):** Helps you recover after a major failure or disaster (e.g. region outage).
- HA focuses on uptime, DR focuses on recovery.
- Both are essential for business continuity.

"HA prevents downtime. DR ensures you can recover when the unexpected happens."

2. Multi-AZ vs Multi-Region

Multi-AZ (High Availability)



- Resources are deployed across multiple Availability Zones in the same region.
- Protects against AZ failures (power, network, etc).
- Lower latency and cost compared to multi-region.
- Examples: RDS Multi-AZ, ALB, Auto Scaling.

Multi-Region (Disaster Recovery)



- Resources are deployed across different regions.
- Protects against regional disasters (e.g. natural disaster, large-scale outage).
- Higher cost and more complex to manage.
- Used for DR or global applications.

3. DR Strategies

Pilot Light



- Minimal infrastructure in secondary region.
- Quick to scale when needed.
- Lower cost.
- RTO: Hours
- RPO: Hours

Warm Standby



- Scaled-down version of production.
- Faster recovery than pilot light.
- Medium cost.
- RTO: Tens of minutes
- RPO: Minutes

Active-Passive



- Primary region active, secondary on standby.
- Automatic or manual failover.
- Higher cost.
- RTO: Minutes
- RPO: Minutes (with replication)

Active-Active



- Both regions active.
- Traffic distributed globally.
- Highest cost and complexity.
- RTO: Seconds
- RPO: Near zero



- Regularly back up critical data (EBS, RDS, DynamoDB, S3, etc).
- Store backups in a different region or account.
- Test restores regularly.
- Use lifecycle policies for backup retention.
- Ensure backups are encrypted.

A backup is not a recovery plan until you test it.

5. RTO and RPO



RTO (Recovery Time Objective)
• Maximum acceptable downtime.
• Example: 1 hour.



RPO (Recovery Point Objective)
• Maximum acceptable data loss.
• Example: 5 minutes.

Define RTO and RPO based on business impact, not technical preference.

6. Dependency Mapping



- Identify all dependencies (databases, APIs, queues, networks, IAM, etc).
- Understand the order of recovery.
- Map cross-region dependencies (e.g. external services, DNS, certificates).
- Document and keep updated.

You cannot recover what you do not understand.

7. Failover Process



- 1 Detect the issue.
- 2 Validate the outage (avoid false failover).
- 3 Trigger failover (manual or automatic).
- 4 Update DNS or routing (e.g. Route 53).
- 5 Verify application health.
- 6 Communicate with stakeholders.

8. Recovery Testing



- Test your DR plan regularly.
- Validate RTO and RPO targets.
- Test backups and restore procedures.
- Run game days and chaos engineering exercises.
- Document results and improve.
- Involve all teams (infrastructure, application, security, business).

A DR plan that is not tested will fail when you need it most.

9. Common AWS Services for HA and DR



EC2
Auto Scaling
Multi-AZ



RDS
Multi-AZ
Read Replicas



S3
Cross-Region
Replication



Route 53
DNS Failover
Health Checks



AWS Backup
Automated
Backups



CloudWatch
Monitoring
and Alarms

10. Best Practices

- ✓ Design for failure from the beginning.
- ✓ Use Multi-AZ for critical workloads.
- ✓ Implement cross-region DR for important data.
- ✓ Automate backups and test restores.
- ✓ Keep infrastructure as code (Terraform/CloudFormation).
- ✓ Use health checks and automated failover.
- ✓ Regularly test your DR plan.
- ✓ Document everything.
- ✓ Review and update RTO/RPO regularly.

11. Key Takeaways



- HA keeps you running.
- DR gets you back online.
- Choose the right DR strategy for your business needs.
- Define and test RTO and RPO.
- Understand and map dependencies.
- Automate and regularly test recovery.
- Be prepared before a disaster happens.



Remember: Disasters are not a matter of if, but when. Plan, test and be ready.



Pro Tip: Start small, test often, and continuously improve your HA and DR strategy.

AWS Cost, Quotas, and Scaling Limits

Cost allocation | NAT costs | Data transfer | Idle resources | Savings Plans | Spot | Service quotas | API throttling | Connection limits | Capacity limits | Cost versus resilience decisions

1. Why Cost, Quotas and Limits Matter



- Helps you run reliable, scalable and cost-effective AWS environments.
- Prevents unexpected bills and service disruptions.
- Helps you plan capacity and avoid limits.
- Forces you to make the right trade-offs between cost and resilience.
- Essential for production readiness and business continuity.

"Control cost. Respect limits. Design for resilience."

2. Cost Allocation and Visibility



- Use AWS Cost Explorer to analyze spending.
- Implement cost allocation tags (e.g. env, team, project).
- Use AWS Budgets for alerts and thresholds.
- Create separate accounts or cost centers for different workloads.
- Use AWS Cost and Usage Reports (CUR) for detailed analysis.
- Regularly review and remove unallocated costs.

3. NAT Costs and Data Transfer



- NAT Gateways are convenient but expensive (hourly + data processing).
- Consider alternatives: VPC endpoints (S3, DynamoDB), NAT instances (for lower cost).
- Monitor data transfer costs (especially cross-region and internet egress).
- Keep resources in the same region and Availability Zone when possible.
- Use CloudFront to reduce internet data transfer costs.

4. Idle Resources (Hidden Costs)



- Identify and remove unused resources (EC2, EBS, RDS, Load Balancers, Elastic IPs).
- Use AWS Compute Optimizer for recommendations.
- Stop non-production environments when not in use.
- Clean up old snapshots, AMIs and logs.
- Use lifecycle policies to automatically remove old resources.

Idle resources waste money. Automate cleanup.

5. Savings Plans and Spot

Savings Plans



- Commit to 1 or 3 years for lower compute costs.
- Covers EC2, Fargate and Lambda.
- Best for stable, predictable workloads.
- Offers up to 72% savings compared to on-demand.

Spot Instances



- Use spare AWS capacity at significant discounts (up to 90%).
- Ideal for stateless, fault-tolerant workloads (batch, CI/CD, big data).
- Can be interrupted with short notice.
- Use Spot with Auto Scaling and multiple instance types.

6. Service Quotas



- AWS has default quotas for each service (e.g. EC2 instances, VPCs, Load Balancers).
- Check quotas in the Service Quotas console.
- Request quota increases when needed.
- Plan ahead for large-scale deployments.
- Some quotas are regional, others are account-wide.

Know your limits before you hit them.

7. API Throttling and Rate Limits



- AWS APIs have request rate limits (e.g. DescribeInstances).
- Exceeding limits causes throttling errors (ThrottlingException).
- Use exponential backoff and retries.
- Monitor API usage with CloudWatch.
- For high-scale automation, use batching and caching.

8. Connection and Capacity Limits



- Services have connection limits (e.g. RDS connections, ELB targets).
- Databases have max connections (consider connection pooling).
- Load balancers have target limits.
- Lambda has concurrency limits (default 1,000 per region).
- Plan for capacity limits in your design and test under load.

9. Cost vs Resilience Decisions



- Multi-AZ increases cost but improves availability.
- Multi-region provides higher resilience but higher cost.
- Use Spot for non-critical workloads, on-demand for critical workloads.
- Choose the right storage class (e.g. S3 Standard vs Glacier).
- Balance RTO/RPO requirements with your budget.

Higher resilience costs more. Choose the right level for your business needs.

10. Common Issues and Solutions

Issue	Cause	Solution
Unexpected high bill	Untracked resources	Use cost allocation tags and budgets
High NAT costs	NAT Gateway and data transfer	Use VPC endpoints or NAT instances
Service limit reached	Default quota too low	Request quota increase
API throttling errors	Too many API requests	Use retries with exponential backoff
Database connection errors	Max connections reached	Use connection pooling
Spot interruption	Instance reclaimed	Use Auto Scaling and diversify instance types
Idle resources	Left running	Automate stop/terminate with policies

11. Best Practices

- Use tagging and cost allocation from day one.
- Monitor costs and set up budgets with alerts.
- Remove or stop idle resources regularly.
- Use Savings Plans for predictable workloads.
- Use Spot for fault-tolerant workloads.
- Check and request service quotas early.
- Implement retries and backoff for API calls.
- Use connection pooling for databases.
- Design for right balance of cost and resilience.
- Regularly review and optimize your architecture.



Remember: Quotas, costs and limits can cause outages, application failures and unexpected bills. Monitor and plan early.



Pro Tip: Automate cost monitoring, resource cleanup and quota checks as part of your DevOps workflow.

AWS Production Incident Command Center

A Senior Troubleshooting Guide — Check | Investigate | Mitigate | Recover | Prevent

Stay calm. Be systematic. Find the real cause. Restore service. Make it stronger.

1. Define Blast Radius



- Identify what is affected (services, regions, users).
- Check customer impact and business criticality.
- Determine if it's partial or full outage.
- Find dependencies (upstream and downstream).

Understand the scope before you make changes.

2. Inspect Recent Changes



- Check recent deployments (code, config, infrastructure).
- Review CloudTrail for API changes.
- Check CI/CD pipelines, manual changes and tickets.
- Look for DNS, security, or network changes.

Many incidents are caused by recent changes.

3. Metrics, Logs and Traces



- Check CloudWatch metrics (CPU, memory, errors, latency).
- Review application logs (CloudWatch Logs).
- Analyze traces (X-Ray) for request flow and errors.
- Look for spikes, anomalies and error patterns.

Let the data guide you.

4. DNS (Route 53)



- Verify DNS resolution (is it working?).
- Check for recent DNS changes.
- Ensure records point to the correct endpoints.
- Check TTL and propagation.

If DNS fails, nothing else matters.

5. Network (VPC)



- Check VPC, subnets, route tables and NACLs.
- Verify security groups and firewall rules.
- Check VPC Flow Logs for rejected traffic.
- Ensure connectivity between components.

No network, no service.

6. Load Balancer (ALB/NLB)



- Check target health status.
- Review access logs.
- Look for 4xx/5xx errors, high latency or dropped connections.
- Verify listener rules and SSL/TLS certificates.

Traffic in, traffic out.

7. Compute (EC2/EKS/Lambda)



- Check instance or task health and scaling.
- Verify resource utilization (CPU, memory, disk).
- Look for crashes, restarts or OOM errors.
- Check if auto scaling is working.

Is your application running?

8. Application



- Check application logs and error messages.
- Verify environment variables and configuration.
- Look for recent releases or feature flags.
- Confirm external service dependencies.

Application errors often explain the symptoms.

9. Database and Dependencies



- Check RDS/Aurora/other DB health (connections, CPU, locks).
- Review slow queries and replication lag.
- Check dependent services (ElastiCache, SQS, third-party APIs).
- Verify storage, IOPS and connection limits.

A healthy application needs healthy dependencies.

10. IAM and Permissions



- Check for permission errors in logs.
- Verify IAM roles and policies have not changed.
- Look for expired credentials or failed authentication.
- Review recent IAM changes in CloudTrail.

Access issues can look like outages.

11. Quotas and Limits



- Check service quotas (EC2, ELB, RDS, etc.).
- Look for API throttling errors (ThrottlingException).
- Verify connection limits (database, load balancer).
- Check account and regional limits.

You might be hitting a limit, not a bug.

12. Mitigate and Recover



- Apply quick mitigation (e.g. rollback, scale, failover).
- Use runbooks and tested procedures.
- Communicate with stakeholders.
- Restore full service and monitor closely.

Stop the bleeding, then restore properly.

13. Verify and Validate



- Confirm the service is fully restored.
- Monitor metrics, logs and user reports.
- Check for secondary issues.
- Ensure no unexpected side effects.

It's not over until it's stable.

14. Prevent Recurrence



- Document root cause and timeline.
- Implement permanent fixes.
- Add monitoring and alerts.
- Improve runbooks and tests.
- Review processes to avoid similar issues.

Turn every incident into a lesson.

Common AWS Tools

	CloudWatch	Metrics, logs, alarms
	CloudTrail	API activity and changes
	X-Ray	Traces and request flow
	Route 53	DNS and health checks
	VPC Flow Logs	Network traffic
	Systems Manager	Instance access and diagnostics
	AWS Health	Service health and events
	Service Quotas	Limits and quota information

Incident Command Checklist

- ✓ Identify blast radius
- ✓ Check recent changes
- ✓ Analyze metrics, logs and traces
- ✓ Verify DNS
- ✓ Check network and security
- ✓ Inspect load balancer
- ✓ Validate compute resources
- ✓ Review application and dependencies
- ✓ Confirm IAM and quotas
- ✓ Mitigate, recover and verify
- ✓ Document and prevent recurrence



Remember: Incidents happen. Good engineers are not the ones who never fail, but the ones who recover, learn and improve.



Pro Tip: Build runbooks, automate recovery where possible, and regularly practice incident scenarios.